零基础实习手册 / Windows 与 Linux

计算机网络

零基础入门与实践

从网络基础到实习项目

DNS 查地址

TCP 与 TLS 建连接

HTTP 取内容

按章节学习

点击条目跳转

先认识原理

01 从打开网页认识网络 02

02 局域网、交换机与 ARP 03

03 IPv4 地址与子网 04

04 路由、网关、DHCP 与 NAT 05

05 端口、TCP 与 UDP 06

06 DNS、HTTP 与 HTTPS 07

再动手验证

A 查看自己的网络配置 08

B 分段测试连通性 09

C 分别验证 DNS 和 HTTP 10

D 在本机搭一个 HTTP 服务 11

E 用 Wireshark 看握手与请求 12

F 用 Python 写一个回显通信 13

排障与复习

按现象缩小故障范围 14

自测与动手验收 15

自测答案与学习顺序 16

从这里开始

先读第 2 页，再做实验 A（第 8 页）。第一次只要找到 IP 地址和默认网关。

求职路线与项目 17

从打开网页认识网络

读完能把一次网页访问拆成几步，并说出每一层负责什么。

网络是一组能交换数据的设备。协议是双方约定的规则，例如数据怎么写、怎样识别响应、出错时如何处理。浏览器是客户端，提供网页的程序是服务器。这两个词描述程序在一次交互中扮演的角色。

以浏览器访问一个新站点为例

1. 查地址。浏览器需要知道该往哪个 IP 地址发数据，通常会通过 DNS 查询域名。若已有缓存，可能省去实际查询。
2. 建立通信。以常见的 HTTPS over TCP 为例，先建立 TCP 连接，再完成 TLS 握手。已有连接时也可能直接复用。
3. 交换内容。浏览器发送 HTTP 请求，服务器返回状态码、响应头和可能的响应体。页面中的图片等资源还可能触发额外请求。

用五层模型理解分工

层次	负责什么	常见例子
应用层	规定程序之间的消息含义	HTTP、DNS
传输层	为端点上的应用提供传输服务	TCP、UDP、端口
网络层	跨网络把 IP 包送往目的地	IP、ICMP、路由
数据链路层	在一段链路上运送帧	以太网、Wi-Fi、MAC
物理层	把比特表示成可传输的信号	电信号、光信号、无线电波

发送时加上控制信息，接收时逐层解析

以 HTTP 经 TCP 和以太网传输为例，数据会依次装进 TCP 报文段、IP 包和链路层帧。路由器转发时通常会用新的链路层帧发送 IP 包。一条 HTTP 消息可能跨多个 TCP 报文段，一个 TCP 报文段也可能含有多条应用消息的部分内容。

> 第一遍先认清这五层。遇到一个术语，就问它在解决哪一步的问题。以后做实验时，用抓到的真实数据回看这一页。

局域网、交换机与 ARP

读完能解释 IP 地址和 MAC 地址各用在什么地方。

同一个以太网局域网中的设备用帧交换数据。帧中有源 MAC 和目的 MAC。MAC 地址用来标识链路上的接口，常见写法如 02:00:00:00:00:0a。

交换机怎样送帧

1. 收到帧后，交换机用源 MAC 记住这个地址来自哪个端口。
2. 目的 MAC 已知时，交换机按表转发到相应端口。未知单播通常在同一广播域内从其他相关端口泛洪。广播帧也会在广播域内传播。
3. 家里的无线路由器常集成了 Wi-Fi 接入点、交换机和路由器功能。学习时可以按功能分别理解。

知道 IP，怎样得到下一跳的 MAC

假设 A 的地址是 192.168.1.10/24，B 是 192.168.1.20/24，两台机器位于同一以太网广播域。A 判断 B 在本地链路上。若 ARP 缓存没有对应记录，A 广播询问「谁拥有 192.168.1.20」，B 通常以单播回自己的 MAC，A 就可以向 B 发送帧。

目的 IP 的位置	通常要解析的 MAC
与自己直连的以太网	目标设备的 MAC
远端网络，经由网关	下一跳网关的 MAC

当目的地在远端时，这一跳的帧写网关的 MAC，帧里的 IP 包写远端设备的目的 IP。网关收到后继续转发。IPv6 在这类场景用邻居发现（NDP）完成地址解析。

检查自己的理解。若设备访问远端网站，它在家里的以太网上通常会 ARP 查询谁的 MAC？答案可在第 16 页核对。

IPv4 地址与子网

读完能算一个简单的网络地址，并判断两个地址是否落在同一前缀里。

IPv4 地址共 32 位，分成四组，每组 8 位，用 0 到 255 的十进制数表示。斜杠后面的数字是前缀长度，表示前多少位是网络前缀。/24 的掩码为 255.255.255.0，/26 的掩码为 255.255.255.192。

/24 可以先看前三组

192.168.1.10/24 的网络地址是 192.168.1.0。192.168.1.20 在同一前缀内，192.168.2.20 在前缀外。一般做法是把 IP 与掩码逐位做 AND，得到网络地址。

/26 怎样心算

1. /26 的主机位有 6 位，因此每块有 2^6 = 64 个地址。同一个 /24 的末组从 0、64、128、192 开始分块。
2. 192.168.1.70/26 的末组 70 落在 64 到 127，因此网络地址是 192.168.1.64，广播地址是 192.168.1.127。
3. 这个普通广播子网的可分配范围是 192.168.1.65 至 192.168.1.126，共 62 个。/31 与 /32 有专门用法，做这类题时单独处理。

先认识几类常见地址

地址或范围	用途
10.0.0.0/8	私有地址范围
172.16.0.0/12	私有地址范围，第二组是 16 至 31
192.168.0.0/16	私有地址范围
127.0.0.1	IPv4 回环地址，发给本机
169.254.0.0/16	IPv4 链路本地地址，限本地链路使用

动手前想一想。192.168.1.62/26 和 192.168.1.65/26 是否落在同一前缀？先写出各自的网络地址，再看第 16 页。

路由、网关、DHCP 与 NAT

读完能区分分配地址、转发数据包和转换地址这三件事。

路由表决定往哪发

主机也有路由表。常见条目包含目的前缀、下一跳和出口接口。直连路由可直接在对应链路上发送。若有多条可用路由匹配，通常优先选择最长前缀。IPv4 默认路由用 0.0.0.0/0 表示，匹配任意 IPv4 目的地址。

示例路由表	下一跳或接口
192.168.1.0/24	本地网卡，直连
10.2.0.0/16	192.168.1.2
0.0.0.0/0	192.168.1.1

发往 10.2.3.4 时，/16 比 /0 更具体，因此示例选 192.168.1.2。发往 192.168.1.20 时走直连。若发往其他目的地且没有更具体的匹配项，才使用默认路由。

DHCP 与 NAT 分别做什么

DHCP 可自动下发 IP 地址、掩码、默认网关和 DNS 等参数。首次获取 IPv4 租约的典型流程是 Discover、Offer、Request、ACK，分别对应发现、提供、请求、确认。

家用路由器常通过 NAT/PAT 把多台内网设备的私有 IPv4 地址及源端口映射为外侧地址及端口，再按映射把响应送回。是否使用 NAT 与路由表怎样选下一跳是两回事。

认识 IPv6

IPv6 地址有 128 位，常用十六进制写成多组。::1 是 IPv6 回环地址，IPv6 默认路由写成 ::/0。双栈网络可以同时使用 IPv4 和 IPv6。做实验时先看清命令实际测试的是哪一种。

> 排障时分别记录「我的地址」「默认网关」「DNS 服务器」。即使三个值碰巧相同或相近，它们承担的职责也不同。

端口、TCP 与 UDP

读完能解释为什么一个 IP 地址可以同时提供多种网络服务。

端口号是 TCP 或 UDP 首部中的编号，取值为 0 至 65535。协议与端口共同参与把收到的数据交给应用。客户端连接服务器时通常使用临时源端口，服务器则在约定的端口上监听。

传输协议	提供的服务	常见应用
TCP	面向连接、可靠有序的字节流	HTTP/1.1、HTTP/2
UDP	保留报文边界，协议本身不保证送达和顺序	常见 DNS 查询、承载 QUIC

TCP 怎样建立连接

1. 客户端发 SYN，提出建立连接并告知自己的初始序号。
2. 服务端回 SYN + ACK，确认客户端的 SYN，并给出自己的初始序号。
3. 客户端回 ACK，确认服务端的 SYN。连接建立后就能交换应用数据。已经建立的连接可以连续传送多个请求。

TCP 的几个高频概念

序号和确认号帮助双方跟踪字节位置，丢失的数据可以通过重传补齐。流量控制考虑接收方能接收多少，拥塞控制根据网络拥塞情况调节发送。正常关闭时，双方可以分别用 FIN 关闭自己的发送方向。

TCP 向应用交付的是字节流。连续调用两次 send，接收方的一次 recv 可能读到部分字节，也可能读到合并的数据。应用需要用长度字段、分隔符等规则确定消息边界。

> ping 常用 ICMP Echo 探测。即使收到 ping 回应，某个 TCP 端口上的程序也可能没有启动。实验 C 和 E 会把这两类结果分开观察。

DNS、HTTP 与 HTTPS

读完能拆解一个 URL，并区分查不到地址、连不上服务和 HTTP 返回错误。

DNS 把名字查成记录

域名系统按名称查询记录。A 记录对应 IPv4 地址，AAAA 对应 IPv6 地址，CNAME 表示别名。客户端常把查询交给递归解析器，解析器有缓存时可直接使用，缓存未命中时再向 DNS 层级查询。TTL 控制记录通常可缓存的时长。

读懂 URL 和 HTTP

URL 示例	https://example.com/notes?id=3
https	方案，说明用 HTTPS 访问
example.com	主机名，HTTPS 的默认端口是 443
/notes	路径
id=3	查询字符串中的一个参数

HTTP 是请求与响应协议。GET 通常用来获取资源。响应状态码 200 表示请求成功，301 表示永久重定向，404 表示找不到请求的资源，500 表示服务器遇到意外错误。

HTTPS 给 HTTP 加上受保护的传输

HTTPS 借助 TLS 提供传输加密、完整性保护和服务器身份验证。浏览器通常检查证书是否受信、有效且匹配域名。HTTP/1.1 和 HTTP/2 的 HTTPS 一般运行在 TCP 上。HTTP/3 运行在 QUIC 上，QUIC 使用 UDP。

> 排查「打不开」时，分别观察名字有没有解析出来、是否连到目标服务、有没有收到 HTTP 状态码。如果同一个请求返回了 404，就从请求路径和服务端处理开始看。

查看自己的网络配置

10–15 分钟 • 只读命令 • 输出一张本机网络参数表

准备

Windows 按 Win + R，输入 cmd 后回车。以下 Windows 命令以 CMD 为例。Linux 打开终端。先使用电脑当前正在联网的 Wi-Fi 或以太网接口。

操作

Windows CMD

```
ipconfig /all
```

Linux 终端

```
ip -br address
ip route
cat /etc/resolv.conf
```

在输出中找这些字段

记录项	示例	从哪看
接口名	Wi-Fi 或 wlan0	当前联网的网卡
IPv4 与掩码/前缀	192.168.1.23/24	IPv4 Address、Subnet Mask 或 ip 输出
默认网关	192.168.1.1	Default Gateway 或 default via
DNS 服务器	192.168.1.1	DNS Servers 或 nameserver

输出会随网络而变。Linux 若显示 127.0.0.53，通常指向本机 DNS 转发服务。在支持 resolvectl 的系统上，可用 resolvectl status 查看上游。

怎样判断完成

写下正在使用的接口、IPv4、前缀、默认网关和 DNS。看到 VPN 或 WSL 虚拟网卡时，先找主机实际联网的接口。若网络只提供 IPv6，可先记录 IPv6 配置，回环实验仍可尝试。

下次提问可直接说「带我做实验 A，我用 Windows」。贴输出时可只贴相关字段，并遮掉公开 IP、完整 MAC 和校园账号等信息。

分段测试连通性

10–20 分钟 • 检查本机回环、默认网关与路径

先测本机，再测网关

从实验 A 找到默认网关。下面的 192.168.1.1 只是示例，运行前换成自己的网关 IP。如果有多个网关，先用实际联网接口对应的那一个。

Windows CMD

```
ping -n 4 127.0.0.1
ping -n 4 192.168.1.1
arp -a
tracert -d example.com
```

Linux 终端

```
ping -c 4 127.0.0.1
ping -c 4 192.168.1.1
ip neigh
traceroute -n example.com
```

读出三个信息

1. ping 的时间是往返时延 RTT。连续 4 次都收到回应，可以记录本轮丢包率为 0%。只有 4 次探测时，结果只代表这次观察。
2. ping 网关后，用邻居表查找网关 IPv4 与链路层地址的对应条目。网关可以回应 ARP，却过滤 ICMP Echo。
3. 路径跟踪用逐步增加的 TTL 探测途经的节点。星号表示在等候时间内没有收到这一跳的探测回应。后续跳或最终目标仍可能回应。-d 或 -n 关闭中间节点 IP 的反向名称解析，命令仍需先解析输入的域名。

某些 Linux 系统未安装 traceroute。可先完成前两项。如果用域名做路径跟踪时先报名称解析错误，转到实验 C 检查 DNS。

分别验证 DNS 和 HTTP

15–20 分钟 • 需要可用的互联网连接 • 找到 A 记录与 HTTP 响应

先查域名

Windows CMD 或 Linux 终端

```
nslookup -type=A example.com
```

在解析结果中找 example.com 对应的一个或多个 IPv4 地址。开头 Server 和 Address 通常描述本次查询使用的 DNS 服务器。缺少 nslookup 的 Linux 环境，可先用 getent ahostsv4 example.com 查看系统解析结果。

再发出 HTTP 请求

Windows CMD

```
curl.exe --max-time 10 -v https://example.com/ -o NUL
```

Linux 终端

```
curl --max-time 10 -v https://example.com/ -o /dev/null
```

-v 显示连接与请求响应的细节。-o 把响应正文送到空设备。输出可能因版本、代理、地址族和网络而异。观察是否解析出地址、是否显示连接、是否出现类似 < HTTP/2 200 或 < HTTP/1.1 200 OK 的状态行。

记录与解释

观察到什么	如何理解
DNS 返回地址	这一轮查询拿到了地址信息
域名解析报错	检查查询名称与本机的 DNS 配置
连接超时	检查目的地址、网络路径或连接过滤
TLS 证书校验失败	核对系统时间、请求域名与证书错误
出现 4xx 或 5xx	本次请求收到 HTTP 层的错误状态

curl 若经由代理，输出中的连接目标可能是代理地址。把这一点记进实验记录。外网访问失败时，实验 D 仍能独立完成。

在本机搭一个 HTTP 服务

15–20 分钟 • 需要 Python 3 • 观察 200、404 和端口未监听

终端一启动服务

建一个空文件夹，进入它，用编辑器保存文件 index.html，内容为 Hello, network!。在 Windows 上确认实际扩展名为 .html。以下服务只绑定本机 IPv4 回环地址。

```
python -m http.server 8000 --bind 127.0.0.1
```

如果 Windows 找不到 python 但安装了启动器，可用 py -3 替换 python。Linux 常用 python3 替换 python。服务启动后保持终端一运行。

终端二发送请求

Windows CMD

```
curl.exe --noproxy 127.0.0.1 -i http://127.0.0.1:8000/index.html
curl.exe --noproxy 127.0.0.1 -i http://127.0.0.1:8000/missing.html
netstat -ano -p tcp | findstr ":8000"
```

Linux 终端

```
curl --noproxy 127.0.0.1 -i http://127.0.0.1:8000/index.html
curl --noproxy 127.0.0.1 -i http://127.0.0.1:8000/missing.html
ss -ltn "sport = :8000"
```

怎样判断完成

1. index.html 请求应返回 200，响应正文含 Hello, network!。不存在的文件应返回 404。终端一会打印访问记录。
2. 在监听列表里找 127.0.0.1:8000。到终端一按 Ctrl + C 停止服务，再请求 index.html，通常会收到连接失败。对比这次错误与刚才的 HTTP 404。

若 8000 已被占用，可换 8001。服务端命令、请求地址和后续抓包过滤器里的端口都要同步修改。需要抓包时，再启动这个服务即可。

用 Wireshark 看握手与请求

20–30 分钟 • 观察实验 D 产生的回环流量

准备

保持实验 D 的服务器运行。在 Wireshark 中选择用于回环捕获的接口。Windows 安装了 Npcap 时，常见名称含有 loopback。Linux 通常选 lo。若没有可用的回环接口或权限，先完成实验 D，之后再做这项。

捕获一次短请求

1. 开始捕获。在另一个终端执行下面的请求，随后停止捕获。每次运行这个 curl 命令会重新发起本次连接。

```
curl.exe --noproxy 127.0.0.1 -i http://127.0.0.1:8000/index.html
```

Linux 把 curl.exe 换成 curl。在 Wireshark 顶部显示过滤器栏输入 tcp.port == 8000，然后回车。

2. 在 Info 栏找 SYN、SYN ACK、ACK。点开一个包的 Transmission Control Protocol，记录源端口、目的端口和标志位。回环通信中双方 IP 都是 127.0.0.1，可以用端口区分两端。
3. 改用 http 显示过滤器，尝试找 GET /index.html 和响应状态。若未识别为 HTTP，可选中相关 TCP 包，用 Follow TCP Stream（追踪 TCP 流）查看明文请求和响应。

可以使用的显示过滤器

写法	效果
tcp.port == 8000	匹配源端口或目的端口为 8000 的 TCP 包
tcp.flags.syn == 1	显示设置了 SYN 标志的 TCP 包
http	显示已解析为 HTTP 的数据包

验收时把三个包的方向写出来，再标出哪一端用了临时端口。若抓不到包，优先检查回环接口、端口号和开始捕获的时机。

用 Python 写一个回显通信

25–35 分钟 · 两个脚本 · 把端口、连接和字节串连起来

保存为 server.py

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.bind(("127.0.0.1", 9000))
    server.listen(1)
    print("Listening on 127.0.0.1:9000")
    connection, address = server.accept()
    with connection:
        message = connection.makefile("rb")
        with message:
            line = message.readline()
        connection.sendall(line)
```

保存为 client.py

```
import socket

with socket.create_connection(("127.0.0.1", 9000), timeout=5) as client:
    client.sendall(b"hello, network!\n")
    with client.makefile("rb") as reply:
        print(reply.readline().decode("utf-8").rstrip("\n"))
```

运行和验收

先在终端一运行 python server.py，再在终端二运行 python client.py。预期客户端打印 hello, network!。服务端只处理一次连接，结束后要重新运行才能再次测试。文件按普通文本保存，注意 Python 缩进。

bind 选定本机地址与端口，listen 开始监听，accept 等待并返回用于和客户端通信的套接字。客户端发的是字节串。这里以换行符标记消息结束，因此两端用 readline 读取一行。可以改消息内容，验证服务端会原样回送。

按现象缩小故障范围

一次改一个条件，记录原命令、原输出和下一步。

现象	先检查	对应实验
找不到可用 IPv4 或默认网关	看当前接口是否已连接、地址是否配置	A
网关 ping 超时	核对网关地址、接口、邻居表，考虑 ICMP 过滤	B
域名解析失败	核对拼写，查看 DNS 服务器与查询结果	C
端口连接失败	看服务是否启动、监听地址和端口是否一致	D
HTTP 404	核对 URL 路径与服务端目录	D
抓包为空	看接口是否选对、开始抓包是否早于请求	E

用实验 D 制造两种可恢复的故障

1. 服务运行时访问 missing.html。收到 404 后，把路径改回 index.html，观察 200。
2. 停止服务后再访问 index.html。记录连接错误。重启服务，再次确认 200。

这两次实验分别在 HTTP 层和 TCP 建连阶段出现问题。保留原错误文本，后面遇到类似报错时就有参照。

速度慢时还要区分三个量

指标	怎么理解
带宽	链路每秒可传输的数据量上限，常用 bit/s
吞吐量	一段时间内实际传过的数据量除以时间
RTT	一次探测发出到收到回应的往返时间

如果链路速率为 100 Mbit/s，换算字节单位约为 12.5 MB/s，实际文件传输通常还会受开销等因素影响。往返时间低与下载吞吐量高是不同的测量结果。

实验记录示例：Windows CMD → 请求本机 8000 端口 → 收到 404 → 查看服务目录 → 改为 /index.html 后返回 200。

自测与动手验收

先独立写下理由，下一页再对答案。

1. 打开一个之前没有访问过的 HTTPS 网站。假设没有缓存和现成连接，并使用 TCP。DNS、TCP、TLS、HTTP 大致按什么顺序工作？
2. 本机访问远端网站时，家里的以太网中通常会用 ARP 查谁的 MAC？同一跳上的目的 IP 指向谁？
3. 192.168.1.62/26 和 192.168.1.65/26 的网络地址分别是什么？两个地址在同一前缀内吗？
4. 路由表有 0.0.0.0/0、10.0.0.0/8、10.2.0.0/16 三项。目的 IP 为 10.2.3.4 时，匹配哪一项？
5. ping 网关成功，但访问某个网站失败。这个结果说明了什么？下一步能做哪两项检查？
6. 本机 HTTP 服务先返回 404，关闭服务后 curl 又报告连接失败。这两种现象分别在哪个阶段发生？
7. TCP 客户端连续发送两段字节，服务端用一次 recv 能否保证完整收到第一段？实验 F 怎样划分消息？
8. 抓包中，两端的 IP 都是 127.0.0.1。看到目的端口是 8000 的初始 SYN 时，谁在发起连接？

动手验收

从一个空目录启动本地 HTTP 服务，用终端完成一次 200、一次 404、一次停服后的连接失败。再用 Wireshark 对一次成功请求标出 SYN、SYN + ACK、ACK 和 GET。最后尝试用 Python 回显程序复现「监听、连接、收发、结束」四个动作。

完成标准是能把命令输出中的具体字段与概念对应起来。例如能指出「这条是目的端口」或「这里已经收到 HTTP 响应」。

自测答案与学习顺序

每轮学习结束后，用一道题和一条实际输出检验理解。

1. DNS 查地址，TCP 建立连接，TLS 握手，HTTP 交换请求和响应。这是题设下的典型顺序。
2. ARP 查下一跳网关的 MAC。帧中的 IP 包仍指向远端目的 IP。网关后续转发时，会为下一跳使用相应的链路层封装。
3. .62 在 0 到 63 这一块，网络地址为 192.168.1.0。.65 在 64 到 127 这一块，网络地址为 192.168.1.64。二者属于不同前缀。
4. 选择 10.2.0.0/16，因为三项均匹配而 /16 最长。
5. 本次 ICMP 探测中网关回应了。下一步可查询域名的 DNS 记录，再用 curl 观察是否能建立连接、通过 TLS 并收到 HTTP 响应。
6. 404 发生在 HTTP 层，说明服务已处理请求并给出状态。停止监听后出现的连接失败发生在 TCP 建连阶段。
7. 无法保证。TCP 提供有序字节流，recv 每次可返回当时收到的一部分字节。实验 F 约定用换行符划分一条消息，并用 readline 读取。
8. 客户端发起。目的端口 8000 是服务端监听端口，客户端通常使用一个临时源端口。

可以分三轮读

顺序	读什么与做什么	做完能解释
第一轮	第 2–5 页，做实验 A、B	本机地址、网关、同网段和跨网段
第二轮	第 6–7 页，做实验 C、D	域名、端口、TCP 和 HTTP 状态
第三轮	做实验 E、F，再做第 15 页自测	用抓包与代码对应连接和应用数据

之后可以直接说「第 4 页 /26 的划分再讲一遍」「带我做实验 A，一次只给一条命令」或「我在实验 D 看到了这个报错」。我可以跟着你的输出一步步解释。

从零基础到可以投递

默认以网络运维、技术支持实习为主线。安全运营作为后续分支。

计算机网络是这几个方向的共同基础。实习投递还需要 Linux 操作、故障记录、脚本能力和一份可以演示的项目。下面按每周约 6–8 小时安排 10 周，进度以能否独立完成任务为准。

周次	学习与操作	当周交付物
第 1–2 周	做实验 A、B。练 IPv4、ARP、网关和子网	本机配置表、两道 /26 计算、一份路径记录
第 3–4 周	做实验 C 至 E。练 DNS、TCP、HTTP 和抓包	一份握手截图和一次 404 / 连接失败的对照
第 5 周	用 Linux 完成实验 G，补做 Python 回显	进程、监听端口与日志的对应记录
第 6–7 周	做项目一，部署 Nginx 并演练两种故障	配置文件、运行手册、故障复盘
第 8 周	做项目二，写 Python 服务巡检脚本	脚本、CSV 示例、三种场景测试结果
第 9–10 周	做陌生故障演练、整理项目与简历	可复现的项目仓库、项目讲解、投递清单

学到哪个阶段可以投

能在 Linux 上独立找到服务的监听端口，读懂一次请求的结果，复现项目一，并用自己的话说明一个故障的定位过程，就可以开始投网络运维和技术支持实习。项目二、抓包和面试练习可以同步补齐。

偏网络工程方向时，增加 VLAN、交换、静态路由和设备配置实验。偏安全运营方向时，增加日志分析和事件初判，先从第 22 页的分支练习起步。

> 第一周先交付一张自己的网络配置表。概念有疑问就带着命令输出提问，后续复盘也用同样的记录方式。

把进程、端口和日志对起来

30–45 分钟 • 在自己的 Linux 环境练习 • 需要 Python 3、curl 和 iproute2

Windows 可使用已有的 WSL Ubuntu，也可以用自己的 Linux 虚拟机。下面所有终端都在同一个 Linux 环境里打开。WSL 的网络配置可能和 Windows 主机不同，做本地服务时先让客户端与服务端都留在 WSL 内。

终端一准备目录并运行服务

```
mkdir -p ~/netlab/www
cd ~/netlab/www
printf 'ok\n' > health.txt
python3 -u -m http.server 8000 --bind 127.0.0.1 2>../backend.log
```

终端二建立证据链

```
cd ~/netlab
ip -br addr
ip route
ss -ltnp 'sport = :8000'
curl --noproxy 127.0.0.1 -i http://127.0.0.1:8000/health.txt
tail -n 5 backend.log
```

在 ss 输出里找监听地址、端口和 PID。PID 是进程号。用 ps -p PID -o pid,user,args 查看该进程，把命令里的 PID 换成实际数字。再从日志找到 GET /health.txt 和 200。

向下一步扩展

用 pwd 确认当前目录，用 ls -l 看文件权限，用 grep 'GET /health.txt' backend.log 过滤日志。2> 把标准错误写到文件，因此 HTTP 访问日志可以从文件读取。

在终端一按 Ctrl + C 停止服务，再运行 ss 和 curl，观察监听条目消失以及请求失败。服务启动失败时，先直接看 backend.log。

验收记录要包含四项：监听地址、进程 PID、HTTP 状态码和对应日志行。能够回答「有进程就一定有端口监听吗」以及「监听 127.0.0.1 是什么意思」。

用 Nginx 部署本地服务

第 6–7 周 • 一个后端、一个反向代理、两份日志 • 上半部分

反向代理接收客户端请求，转发给后端，再将响应回给客户端。本项目让 curl 请求 127.0.0.1:8080，由 Nginx 转发到实验 G 的 Python 服务 127.0.0.1:8000。

准备

在自己的 Linux 环境安装 Nginx，或使用已有安装。运行 nginx -v 确认命令可用。下文使用支持 -e 参数的 Nginx（1.19.5 或更新）。使用实验 G 的目录，保持后端运行。下面用独立的配置、PID 和日志启动实验实例，端口仅绑定在回环地址。

在 ~/netlab 中保存 nginx-lab.conf

```
worker_processes 1;
pid nginx-lab.pid;
error_log nginx-error.log;
events { worker_connections 64; }
http {
    access_log nginx-access.log;
    server {
        listen 127.0.0.1:8080;
        location / {
            proxy_pass http://127.0.0.1:8000;
        }
    }
}
```

用独立配置检查、启动、发请求

```
cd ~/netlab
nginx -t -e "$PWD/nginx-error.log" -p "$PWD/" -c nginx-lab.conf
nginx -e "$PWD/nginx-error.log" -p "$PWD/" -c nginx-lab.conf
curl --noproxy 127.0.0.1 -i http://127.0.0.1:8080/health.txt
```

预期返回 200 和正文 ok。若 8080 已被占用，把配置与后续请求里的 8080 一起换成 8081。后续操作均从 ~/netlab 执行。

官方参考：nginx.org/en/docs/beginners_guide.html。项目使用的 Python 简易 HTTP 服务用于本地练习。

演练 404 和 502 并复盘

用两组请求和两份日志，判断故障出现在哪一段。

先跑通，再一次只改一个条件

```
cd ~/netlab
curl --noproxy 127.0.0.1 -i http://127.0.0.1:8080/health.txt
curl --noproxy 127.0.0.1 -i http://127.0.0.1:8080/missing.txt
tail -n 5 nginx-access.log
tail -n 5 backend.log
```

后端运行时，第一条应得到 200，第二条应得到 404。后端日志里能找到对应路径。确认应用处理了请求之后，从路径和文件开始查。

停后端，保留 Nginx

1. 在后端终端按 Ctrl + C，重新请求代理的 /health.txt。通常会收到 502。直接请求 8000 则会连接失败。
2. 查看 ss -ltnp 的输出，核对 8080 在监听、8000 已停止。运行 tail -n 5 nginx-error.log，看代理连接上游失败的记录。
3. 按实验 G 重启后端。通过代理再请求一次 /health.txt，确认恢复 200。记录从发现故障到验证恢复所用的步骤。

停掉实验实例

```
cd ~/netlab
nginx -e "$PWD/nginx-error.log" -p "$PWD/" -c nginx-lab.conf -s quit
```

可以放进仓库的内容

放入 nginx-lab.conf、示例 health.txt、README 和一篇故障复盘。README 要写清启动顺序、两个端口、测试命令、预期结果和停止方法。复盘用「现象、假设、证据、修复、验证」五项写清，示例日志只保留相关行。

演示验收：让同学按 README 运行一次。你能边演示边回答「怎样判断 502 来自代理」「为什么直连 8000 与访问 8080 的报错不同」。

写一份 Python 服务巡检脚本

第 8 周 · 用你学过的文件操作把每次检查保存成 CSV

输入与输出

使用 Python 标准库。第一版只检查自己的本地服务，固定使用项目一的两个地址。每次检查输出时间、URL、HTTP 状态码、耗时毫秒、结果和错误说明。CSV 可用表格软件打开。脚本每次执行检查一轮。

输出字段	示例
timestamp、url	检查时间、http://127.0.0.1:8080/health.txt
status、elapsed_ms	200、3.42
result、detail	OK、HTTP_ERROR 或 CONNECT_ERROR，另存错误说明

分三步写

1. 用 urllib.request.build_opener 创建请求器，传入 urllib.request.ProxyHandler({}) 以直接访问本机。调用请求器的 open 请求固定 URL，设置 timeout=3。用 time.perf_counter() 在请求前后计时，差值乘以 1000，得到毫秒。耗时定义为发出请求到收到响应头或发生异常。
2. 先捕获 urllib.error.HTTPError，记录其 code。再处理 urllib.error.URLError 和 TimeoutError，HTTP 状态留空，把具体异常类型和原因写进 detail。
3. 用 csv.DictWriter 把结果追加到 checks.csv。新文件或空文件先写表头，用 newline="" 打开文件。用 with 关闭正常响应和 HTTPError 响应。每次运行打印一行汇总。

三个必测场景

场景	预期输出
两个服务都运行	两条 200，结果为 OK
改为访问不存在的路径	状态 404，结果为 HTTP_ERROR
保留代理并停掉后端	直连记录连接错误，代理记录 502

交付 check_http.py、README 和一份实际生成的 checks.csv。讲解时指出一行记录对应哪次请求，以及为何耗时是客户端测量值。

按实习方向补一项技能

完成两个项目后，用一项贴合目标岗位的练习深化。

网络运维、网络工程

在网络模拟器里建一个小拓扑。两台主机接同一交换机，分别放进 VLAN 10 和 VLAN 20。先观察二层隔离，再接入三层网关完成跨 VLAN 转发。最后给两侧主机各画一条通信路径。

练习步骤	可交付证据
划分两个 VLAN	拓扑图、两侧 IP 前缀与端口分配表
配置网关并验证	跨 VLAN 连通结果、网关路由表
故意填错一台主机的默认网关	原错误、修复步骤、重测结果

技术支持

从项目一选一个故障，写一份给同事看的工单。记录影响范围、起止时间、用户报错、已验证的层次、证据和下一步。如果需要移交，把别人能直接执行的复现步骤写在前面。

安全运营

使用自己的实验访问日志，统计各路径的 2xx、4xx 和 5xx 数量。拿一次已知的 404 实验对照时间和请求路径。写出观察、可能解释和需要补充的证据。单个 404 只能说明一次资源请求没有找到对应内容。

选方向时，用三条真实岗位描述做表。把要求写成「岗位原文、我能演示的证据、还缺的技能」三列，再决定先补哪个练习。

不看教程做一次排障

给自己 25 分钟，先写判断依据，再运行下一条命令。

题目一：代理可访问，但返回 502

先记录代理 URL、时间和状态码。检查 8080 与 8000 的监听状态。分别请求代理和后端，对照 Nginx 错误日志。若后端端口没有监听，再检查后端进程和启动日志。修复后从原入口重测。

题目二：域名访问失败，已知服务 IP 可以连接

对照实验 C，核对请求域名和 DNS 查询结果，记录使用的解析器。HTTPS 访问还涉及域名与证书匹配。可以继续用正确域名观察 curl 的连接及 TLS 输出，确定失败发生在解析、建连、证书校验或 HTTP 哪一步。

题目三：同一交换机上的两台主机不通

核对接口状态、IP、掩码和 VLAN。两台主机应在同一二层网络且 IP 前缀一致时，可观察 ARP 是否得到回应。确认 ARP 成功后，再解释 ICMP 的结果。

面试口答练习

1. 三次握手中哪一端先发 SYN？怎样用源端口和目的端口验证？
2. DNS 返回了 IP，为什么浏览器还可能打不开网页？按连接、TLS、HTTP 各举一项检查。
3. 什么时候应检查网关，什么时候应检查本机监听端口？各举一个实际场景。
4. 你怎样证明服务恢复了？说明检查请求、预期响应及对应日志。

自评用三项：命令用对、解释与输出一致、修复后重测。说出结论时，顺手指出支撑结论的那一行输出。

把实验写成可验证的项目

以自己实际完成的内容为准，把命令和证据留在仓库。

简历项目写法

项目名可写「Linux 本地 HTTP 服务部署与排障」。描述示例：用 Nginx 为 Python HTTP 服务配置反向代理，通过 curl、ss 和访问日志区分 HTTP 404、代理 502 与端口未监听，并整理可复现的部署和排障说明。

第二条可写「用 Python 实现 HTTP 服务巡检，将状态码、响应耗时和连接异常写入 CSV，并验证正常请求、路径错误与后端停止场景」。完成脚本和测试后再使用这一条。

投递前逐项确认

1. 能用 90 秒介绍项目目标、结构、一次故障和如何验证修复。
2. 在干净的实验目录照 README 重做项目，能独立找到服务日志和监听端口。
3. 能算一个 /26 子网，说明 DNS、TCP、TLS 和 HTTP 的先后关系，能从抓包里认出握手。
4. 能把一次排障过程写成半页记录，别人照着命令可以复现。
5. 用目标岗位的原始要求对照项目证据，写明自己已经做过的事。

可以使用的岗位关键词

岗位方向	优先展示的证据
网络运维实习	Linux 命令、路由基础、服务部署、故障复盘
技术支持实习	分层排障、工单记录、验证和交接
安全运营实习	HTTP 与 DNS 基础、日志统计、事件初判

可从网络运维和技术支持的寒暑假实习开始投递。岗位要求和你的项目证据有交集时，就可以一边面试一边补薄弱环节。